

Mikiri: Knowing When a Frozen Go Engine Has Searched Enough

Anonymous submission

Abstract

A Go engine in the AlphaZero family gives every move the same search budget. We study how much stronger a frozen engine becomes when only that allocation changes. Mikiri wraps KataGo, leaves its network and search untouched, and climbs a short ladder of visit budgets, stopping at the first rung where the winrate still at stake no longer pays for the next rung. We build a dataset of 6,000 positions searched at seven budgets, with every candidate move scored by an independent forced search, and use it to train the stopping rule, to forecast match strength, and to compare against ten published stopping rules re-implemented on the same engine. Offline, Mikiri matches uniform search at 1.8 to 2.1 times the visits, ahead of the best prior rule by 0.54 [0.28, 0.84] in a paired test. In play against KataGo at 200 visits per move with the same network, it gains +263 Elo (95% interval +230 to +301) at equal visits over 600 games, where doubling KataGo’s own budget gains +237. Dividing the predicted stake by the price of the next rung is the decisive ingredient: it turns stopping signals that are worse than no gate into gains, and it improves prior learned rules as well. We also report two negative results: approximate retrieval of stored searches does not transfer between similar positions, and an exact search memory saves visits but almost no network evaluations.

1 Introduction

Engines in the AlphaGo line choose a move by a tree search guided by a policy and value network [Silver et al., 2016; Silver et al., 2017], and they are compared at a fixed number of search visits per move. KataGo [Wu, 2019], the strongest openly available engine of this kind, follows the convention: at 200 visits per move, a forced recapture and a whole-board fight receive the same 200 visits.

How much does that uniformity cost? Classical work on time management for Monte-Carlo tree search reports large gains from spending more time on some moves than on others [Huang et al., 2010; Baier and Winands, 2016]. For neural engines the evidence comes from small boards and small networks [Lan et al., 2021; Ye et al., 2022; Muppidi et al., 2026], and KataGo already adapts its search inside the tree [KataGo project, 2026]. Whether a per-move budget decision still helps a modern, strong engine on the full board is an open empirical question.

We answer it with a wrapper, Mikiri, that changes nothing but the number of visits a move receives. It searches at 50 visits and asks a small model whether the decision is settled. If not, it searches at the next rung of a ladder, and asks again. The model predicts the winrate the current choice may still be giving up, and search stops when that stake, divided by the visits the next rung would add, falls below a threshold.

Our contributions are these.

- A testbed with exact accounting. Both players use the same engine, network and move-selection rule. Compute is reported as visits, as visits with every restarted search charged, and as network evaluations measured per player.
- A ladder dataset with independent labels. 6,000 positions, seven budgets each, every chosen move scored by a separate forced search. It shows that search error is rare and concentrated, and it forecasts match Elo.
- A stopping rule that is worth a doubling of search. +263 Elo over KataGo at equal visits, and +93 Elo with 20% fewer.
- A comparison with ten published rules on one engine, offline and, for the strongest two, in play.
- The rate rule. Pricing the next rung matters more than the model, and improves prior signals.
- Two negative results on retrieval for a frozen engine.

2 Related Work

Deciding how long to search. Huang et al. [2010] extend search when the most-visited move is not the highest-valued one. Baier and Winands [2016] find that stopping

early when the leader cannot be overtaken is the most reliable of several strategies, and Leela Chess Zero ships a variant of it together with a stop on the KL gain of the visit distribution [Leela Chess Zero contributors, 2026]. Hay et al. [2012] and Kaufmann and Koolen [2017] give stopping rules with a value-of-information and a best-arm-identification rationale. For AlphaZero-style agents, DS-MCTS [Lan et al., 2021] trains networks to predict whether the current choice will be worse than the final one, V-MCTS [Ye et al., 2022] stops when virtually expanded policies have converged, and Muppidi et al. [2026] learn a budget chosen before search. These neural results are on 9×9 boards or small planners. We test all of them on 19×19 with a strong network (Section 6.1).

Reusing search. Transposition tables and graph search share statistics inside one search [Childs et al., 2008; Czech et al., 2021]; opening books do so across games [Wu, 2026]. Retrieval of similar positions helps agents whose networks are trained to read the neighbours [Humphreys et al., 2022; Goyal et al., 2022] and inside a single search [Xiao et al., 2018]. We ask what a frozen engine can reuse.

3 Testbed

Engine and rules. Both players query KataGo v1.18.2 through its analysis engine. Each query is an independent search with an exact visit limit and one search thread, so no tree is inherited between moves and a visit means the same thing at every budget. The network is the 18-block b18c384nbt unless stated. Games use area scoring, positional superko and komi 7, the value at which the engine rates the empty board as even. Both players select moves as KataGo does by default, sampling in proportion to visits $^{1/T}$ with T decaying from 0.5 to 0.1. A player resigns after three consecutive turns below 3% winrate.

Compute. Mikiri searches a position at 50 visits and then perhaps at 200. With one thread and cached evaluations the shallow tree is a prefix of the deep one: on fresh engines, a 200-visit search followed by a 400-visit search evaluated 366 to 396 network rows, against 353 to 395 for the 400-visit search alone. We nevertheless report three measures: visits, the size of the deepest search per move; restart visits, the sum of all searches requested; and evaluations, network rows executed by each player’s own engine process. Budgets are matched on visits.

Matches. A match is a series of games from the empty board with alternating colours. Intervals are bootstrap percentiles over colour-balanced pairs. KataGo against itself at 200 visits scores 50.2% over 400 games (+2 Elo).

4 The Ladder Dataset

We played 300 self-play games at budgets drawn log-uniformly from 30 to 200 visits and sampled 20 positions per game. Each position is searched at 50, 100, 200, 400, 800, 1,600 and 3,200 visits. A search cannot grade itself, so for every distinct move any rung chose we run a

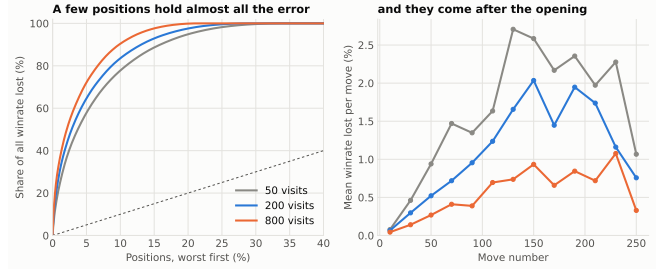


Figure 1: Left: share of all regret held by the worst positions. Right: mean regret by move number.

Signal	50	200	800
Top move’s visit share	0.75	0.74	0.74
LCB margin	0.74	0.73	0.73
v1 entropy gate	0.73	0.73	0.74
KL(visits, prior)	0.65	0.63	0.61
Most-visited is not best-valued	0.60	0.59	0.61
KataGo’s predicted winrate error	0.58	0.58	0.60
Searched minus raw winrate	0.57	0.56	0.56

Table 1: AUC for ranking positions by remaining regret above 0.5%, from the search at 50, 200 and 800 visits.

separate 1,600-visit search restricted to that move at the root. Its winrate $q(s, a)$ is our value of playing a . The regret of stopping at rung k is $r_k(s) = \max_a q(s, a) - q(s, a_k(s))$, where a_k is the move chosen at rung k and the maximum is over scored moves. For a policy with mean regret $\bar{r}$ we report the uniform budget with the same mean regret, interpolating the uniform curve in log visits. The ratio of that budget to the policy’s own mean cost is its compute multiplier.

Regret is rare and concentrated. At 200 visits the engine gives up 0.99% of winrate per move, and 71% of positions carry no regret. The worst tenth hold 84% of it (Figure 1). Regret averages 0.19% per move over the first 40 moves and 1.4 to 1.6% after move 100. A stopper that knew the true regret would reach a multiplier of 12 at a 200-visit budget.

No single signal finds it. Table 1 asks how well individual search statistics rank positions by whether more than 0.5% of winrate remains on the table. Measures of how contested the root is reach an AUC of about 0.74. KataGo’s own prediction of its short-term winrate error, which it uses inside the tree to weight playouts, reaches 0.58 to 0.60. The classical cue that the most-visited move is not the highest-valued one [Huang et al., 2010] reaches 0.59 to 0.61. Contested positions are common, and most are contested between moves of nearly equal value, where being wrong costs nothing.

5 Mikiri

The rate rule. Let the ladder have rungs $v_1 < \dots < v_m$, and let a stopping policy π stop position s at rung $\pi(s)$. We want the policy that minimises expected regret

How Mikiri decides one move

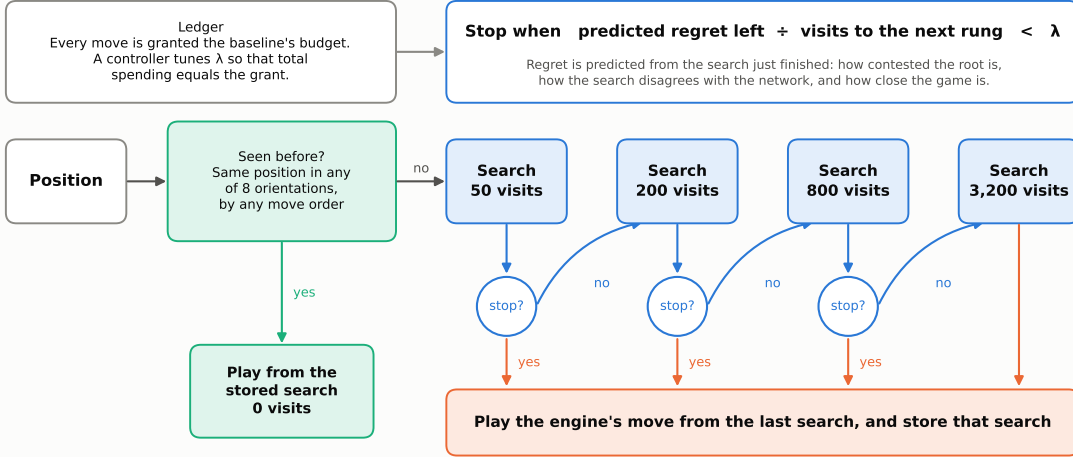


Figure 2: How Mikiri decides one move.

subject to an expected cost,

$$\min_{\pi} \mathbb{E}[r_{\pi(s)}(s)] \quad \text{s.t.} \quad \mathbb{E}[v_{\pi(s)}] \leq B. \quad (1)$$

With a multiplier λ on the constraint, the problem separates over positions: each position minimises $r_k(s) + \lambda v_k$. At rung j the visits already spent cannot be recovered, and continuing by one rung is worthwhile exactly when $\mathbb{E}[r_j - r_{j+1} | x] > \lambda(v_{j+1} - v_j)$, where x is what the finished search shows. The reduction $r_j - r_{j+1}$ is at most r_j . Replacing it by a prediction $\hat{r}_j(x)$ of the regret remaining gives the one-step rule: stop when

$$\frac{\hat{r}_j(x)}{v_{j+1} - v_j} < \lambda. \quad (2)$$

One λ prices every decision on the ladder alike. On a geometric ladder the next rung costs more each time, so going from 800 to 3,200 visits needs four times the stake that going from 200 to 800 does. A flat threshold on $\hat{r}_j$ ignores that price, and spends most of a long ladder's budget on a few positions that no amount of search resolves.

The model. $\hat{r}$ is a gradient-boosted tree ensemble (150 trees of depth 3) fitted to the square root of regret, with all non-final rungs pooled. Its 30 inputs come from the finished search at no engine cost: how contested the root is (visit shares, entropy, the margin between the best move's lower confidence bound and its best rival), how the search disagrees with the network (priors, KL from prior to visits, raw against searched winrate), what is at stake (winrate, distance from 50%, score), KataGo's own raw error predictions, game phase, and what changed since the previous rung. All offline numbers are cross-fitted by game.

The ledge. Mikiri is granted the baseline's budget B for every move, and a ledger records what it spends. The offline frontier gives a map $\lambda(c)$ from a target mean cost

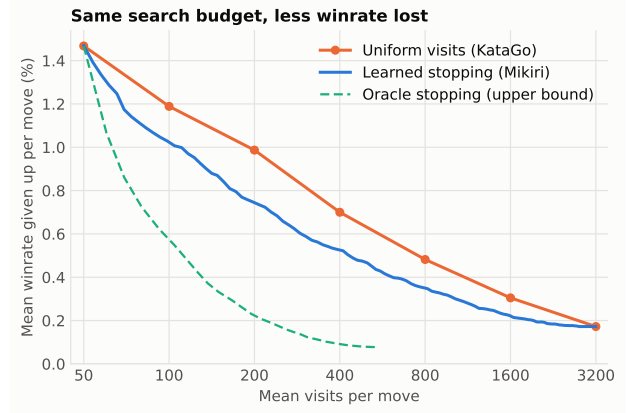


Figure 3: Mean regret against mean visits on the four-rung ladder.

c to a threshold. After each move the controller sets the next target to B plus the ledger's balance spread over a horizon of 3,000 moves, and divides it by a running ratio of realised to commanded spending, which corrects for the difference between the dataset and real games. Spending tracks the grant to within 1% in every match we report.

Memory. Finished searches from the first 80 moves are stored under a key that is invariant to the eight board symmetries and to move order. A later hit is played from the stored search at zero visits.

6 Offline Results

Multiplier. On the four-rung ladder (50, 200, 800, 3,200) the stopper at a mean cost of 200 visits loses as little as uniform search at 358, a multiplier of 1.80, and stays between 1.69 and 1.88 from 100 to 800 visits (Figure 3). With every restart charged it is 1.29 to 1.51. On the full

doubling ladder it reaches 2.13.

What matters. Removing the trajectory features, KataGo’s raw error predictions, or all but seven features changes the multiplier at 200 visits by at most 0.09, and 300 labeled positions already give 1.50. Fitting the square root of regret instead of regret raises it from 1.74 to 1.82. Replacing the rate rule by a flat threshold lowers it to 1.41.

6.1 Comparison with Prior Rules

We re-implemented ten published rules from their papers and source code (Table 2). All see the same root statistics and are scored by the same forced values. BEHIND and UNST search longer when the player is losing or when the most-visited move is not the highest-valued one [Huang et al., 2010]. STOP ends a search once the runner-up cannot overtake the leader in the visits that remain, or in a fraction p of them, and CLOSE extends it when the top two moves are close in visits [Baier and Winands, 2016]. Smart pruning is Leela Chess Zero’s form of STOP, shipped with a stop on the per-visit KL gain of the visit distribution [Leela Chess Zero contributors, 2026]. The VOI rule stops when a bound on the value of further samples falls below their cost [Hay et al., 2012], and the BAI rule when the leader’s confidence interval separates from every rival’s [Kaufmann and Koolen, 2017]. The state-only rule predicts regret from features available before any search, in the spirit of Muppidi et al. [2026]. Two learned rules are adaptations: for DS-MCTS we train a classifier on root statistics to the authors’ label $U(s, n)$, in place of their board-input networks, and for V-MCTS we run the authors’ virtual expansion on KataGo’s root priors, visits and winrates with their published $r = 0.2$ and $\epsilon = 0.1$. Every rule is charged continuation cost, which is what a rule inside the engine pays. Charged for every restart, Mikiri’s four-rung ladder reaches 1.44 at 200 visits, below the 1.60 of V-MCTS, which has no restarts to charge; the matches settle that question in network evaluations.

Three findings. First, the rules with a statistical stopping guarantee are far worse than uniform search here, because they stop on confidence in the leading move and ignore whether the game is still in doubt. Second, V-MCTS is the strongest prior rule (1.60 at 200 visits), and Mikiri exceeds it by 0.54 [0.28, 0.84] at 200 visits and 0.35 [0.09, 0.65] at 400, paired over the same resampled games. Third, the rate rule improves every signal it can be applied to, including the DS-MCTS classifier (1.48 to 1.75), which with it ties Mikiri at 400 visits (0.03 [-0.16, 0.27]).

7 Matches

Equal visits. With the six-rung ladder (50, 200, 400, 800, 1,600, 3,200) Mikiri scores 82.0% (471–87–42), a gain of +263 Elo (+230 to +301). With the four-rung ladder it gains +206 (+181 to +232) over 1,000 games. KataGo at 400 visits gains +237 (Table 3).

Fewer visits and equal evaluations. A fixed-budget engine finds much of its next tree already cached, and

Mikiri’s varying searches reuse less, so it runs more evaluations per visit. Granted 160 visits, it requests fewer visits than KataGo even with restarts charged (197 against 200) and gains +93 Elo (+72 to +114). At equal evaluations the comparison is direct: Mikiri runs 100 per move and gains +263, KataGo at 283 visits runs 98 and gains +108 (Figure 4).

No move sampling. A search that stops when one move dominates has sharper visit counts, which could interact with sampling. With 400 balanced openings each played twice with colours swapped, and both players always playing the top move, Mikiri gains +178 Elo (+153 to +205).

The forecast. Reading the cross-fitted regret at the cost Mikiri actually used, converting to equivalent uniform visits, and reading that budget’s Elo off the measured uniform curve predicts +197 for the four-rung match against the measured +206, +126 at 160 visits against +93, and +225 for the six-rung ladder against +263. The forecast orders the three configurations correctly and is within 40 Elo of each, so the ladder dataset can stand in for matches when choosing among rules.

Prior rules in play. The V-MCTS rule gains ?? Elo (?? to ??) and the DS-MCTS-style classifier ?? (?? to ??), each held to 200 visits by the same controller. The LCB margin with the rate rule and no learning gains ?? (?? to ??), and the entropy gate with a flat threshold ?? (?? to ??).

Where the visits go. Mikiri uses a third of the budget over the first 20 moves and about 270 visits per move around move 140. After move 60 it spends about 130 visits when it rates its winrate above 90% and up to 330 when it is behind but not lost (Figure 5). It scores 76.2% as Black and 77.0% as White. Neither behaviour was programmed; both follow from predicting regret in winrate units.

Other settings. The same model, without retraining, was played in further settings (Table 4).

8 What a Frozen Engine Can Retrieve

Exact memory. Positions recur: across 300 self-play games, a position at move 20 had occurred in an earlier game, up to symmetry, half of the time. In play, memory alone served 21.2% of moves over 600 games with strength unchanged (−9 Elo, −37 to +19) at 158 visits per move. It did not reduce network evaluations (69 per move against 69): KataGo caches evaluations for the life of its process, so the positions memory serves were already cheap.

Approximate retrieval. We embed positions with the raw policy and ownership maps, align over the eight symmetries, and offer the nearest stored position’s 3,200-visit move as a proposal. Where the 200-visit and 3,200-visit moves differ (1,605 positions), the nearest neighbour’s move is right 5.5% of the time, against 43% for the policy network’s own best alternative. A neighbour’s move matches in 96% of cases at similarity above 0.999, which is the same position, and in 27% at 0.90 to 0.97.

Rule	As published, 200	As published, 400	With rate rule, 200
VOI stop [Hay et al., 2012]	0.34 [0.30, 0.40]	0.24 [0.20, 0.31]	1.05 [0.86, 1.21]
BAI stop [Kaufmann and Koolen, 2017]	0.77 [0.63, 1.01]	0.59 [0.50, 0.71]	1.18 [1.09, 1.29]
BEHIND [Huang et al., 2010]	0.89 at 174	0.89 at 348	–
STOP, $p = 0.45$ [Baier and Winands, 2016]	1.25 at 134	1.19 at 512	–
Smart pruning [Leela Chess Zero contributors, 2026]	1.17 at 159	1.21 at 308	–
CLOSE [Baier and Winands, 2016]	1.15 at 229	1.11 at 453	–
UNST [Huang et al., 2010]	1.21 at 239	1.17 at 475	–
KLD gain [Leela Chess Zero contributors, 2026]	1.32 [1.18, 1.49]	1.23 [1.02, 1.42]	–
State only [Muppidi et al., 2026]	1.18 [0.97, 1.40]	1.02 [0.89, 1.23]	1.54 [1.35, 1.74]
DS-MCTS [Lan et al., 2021]	1.48 [1.28, 1.73]	1.48 [1.29, 1.73]	1.75 [1.60, 1.92]
V-MCTS [Ye et al., 2022]	1.60 [1.44, 1.76]	1.54 [1.36, 1.70]	–
LCB margin	1.14 [0.95, 1.36]	0.82 [0.70, 0.97]	1.45 [1.24, 1.70]
Mikiri	1.40 [1.21, 1.63]	1.35 [1.15, 1.61]	2.13 [1.88, 2.41]

Table 2: Compute multipliers on the doubling ladder (above 1 is better than uniform search), with 95% bootstrap intervals over games. Threshold rules are swept and shown at their best point for each budget; rules without a threshold are shown at the configuration nearest the budget, with its cost in visits.

Player (vs. KataGo at 200 visits)	Games	W–L–D	Elo (95% interval)	Visits	Restart	Evals
KataGo, 100 visits	400	77–301–22	–220 (–262, –181)	100	100	40 : 71
KataGo, 283 visits	400	247–126–27	+108 (+76, +143)	283	283	98 : 73
KataGo, 400 visits	400	310–73–17	+237 (+198, +280)	400	400	132 : 73
Mikiri, six-rung ladder	600	471–87–42	+263 (+230, +301)	201	305	100 : 76
Mikiri, four-rung ladder	1,000	728–196–76	+206 (+181, +232)	200	251	99 : 74
Mikiri, four-rung, 160-visit grant	1,000	586–325–89	+93 (+72, +114)	160	197	80 : 74
Mikiri, four-rung, 140-visit grant			(running)			
Mikiri, four-rung, no move sampling	800	561–183–56	+178 (+153, +205)	200	250	115 : 96
V-MCTS rule (Ye et al.)			(running)			
DS-MCTS-style rule (Lan et al.)			(running)			
LCB margin with rate rule, no learning			(running)			
Entropy gate, flat threshold			(running)			
Memory only	600	276–292–32	–9 (–37, +19)	158	158	69 : 69
Mikiri four-rung + memory			(running)			

Table 3: Matches against KataGo at 200 visits per move, 19×19, same network. Visits is the mean size of the deepest search per move, Restart charges every search requested, Evals is network evaluations per move for the player and its opponent.

Opponent and setting	Games	W–L–D	Elo (95% interval)	Visits
KataGo 100 visits, 19x19, b18			(running)	
KataGo 400 visits, 19x19, b18			(running)	
KataGo 800 visits, 19x19, b18			(running)	
KataGo 200 visits, 19x19, b28 network			(running)	
KataGo 200 visits, 13x13, b18			(running)	
KataGo 200 visits, 9x9, b18			(running)	

Table 4: The four-rung stopper, trained once on 19×19 b18 positions, at matched visits in other settings.

gave under 10% of its visits, which bounds what any rule reading root statistics can recover. Our versions of prior rules are re-implementations on a different engine than their authors used. A stopping rule built on the engine’s own statistics offers no protection against adversarial policies [Wang et al., 2023].

10 Conclusion

A frozen Go engine leaves a doubling of search on the table by spending visits uniformly. A small learned stopping rule recovers it, and the part of the rule that matters most is the price it puts on continuing.

Ethical Statement

There are no ethical issues.

Use of AI Assistance

A large language model coding assistant was used extensively in this work: to implement the testbed and

Neighbours’ regret predicts a position’s regret with AUC 0.55.

9 Limitations

Results are for KataGo networks at 100 to 800 visits, against KataGo, where a doubling of visits is worth about 230 Elo; larger budgets are untested. Regret is defined by a search and inherits its blind spots, though the matches do not depend on it. Of the regret that survives a 200-visit search, 57% sits on moves that search

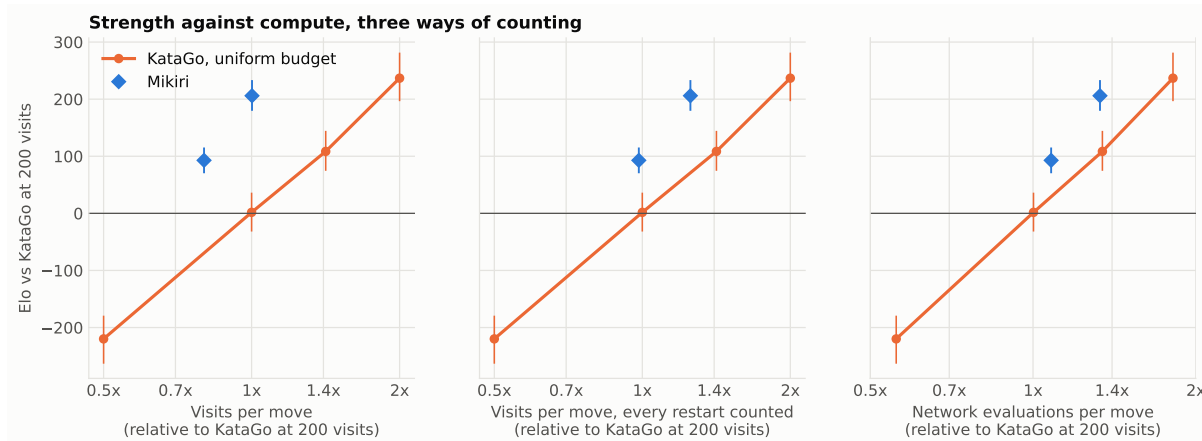


Figure 4: Elo against KataGo at 200 visits as a function of compute relative to that baseline, counted three ways, for the four-rung ladder. The line is KataGo at uniform budgets; bars are 95% intervals.

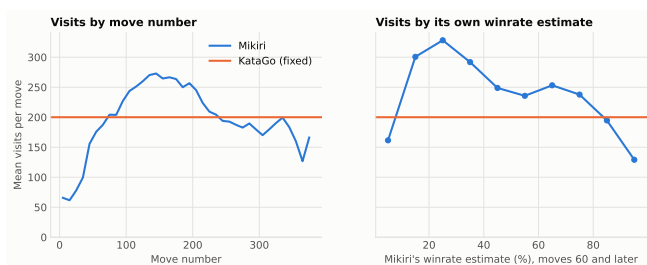


Figure 5: Mean visits per move in the four-rung match, by move number and by Mikiri’s own winrate estimate.

analysis code, to run and monitor the experiments, to produce the figures, and to draft and edit the text. The authors directed the work and are responsible for all of its content. Every reported number is produced by released scripts from released data and game records.

References

- [Baier and Winands, 2016] Hendrik Baier and Mark H. M. Winands. Time management for Monte Carlo tree search. *IEEE Transactions on Computational Intelligence and AI in Games*, 8(3):301–314, 2016.
- [Childs et al., 2008] Benjamin E. Childs, James H. Brodeur, and Levente Kocsis. Transpositions and move groups in Monte Carlo tree search. In *IEEE Symposium on Computational Intelligence and Games*, pages 389–395, 2008.
- [Czech et al., 2021] Johannes Czech, Patrick Korus, and Kristian Kersting. Improving AlphaZero using Monte-Carlo graph search. In *International Conference on Automated Planning and Scheduling*, pages 103–111, 2021. arXiv:2012.11045.
- [Goyal et al., 2022] Anirudh Goyal, Abram L. Friesen, Andrea Banino, Theophane Weber, Nan Rosemary Ke, Adrià Puigdomènech Badia, Arthur Guez, Mehdi Mirza, Peter C. Humphreys, Ksenia Konyushkova,

Laurent Sifre, Michal Valko, Simon Osindero, Timothy Lillicrap, Nicolas Heess, and Charles Blundell. Retrieval-augmented reinforcement learning. In *International Conference on Machine Learning*, pages 7740–7765, 2022. arXiv:2202.08417.

[Hay et al., 2012] Nicholas Hay, Stuart Russell, David Tolpin, and Solomon Eyal Shimony. Selecting computations: Theory and applications. In *Uncertainty in Artificial Intelligence*, 2012. arXiv:1207.5879.

[Huang et al., 2010] Shih-Chieh Huang, Rémi Coulom, and Shun-Shii Lin. Time management for Monte-Carlo tree search applied to the game of Go. In *International Conference on Technologies and Applications of Artificial Intelligence*, pages 462–466, 2010.

[Humphreys et al., 2022] Peter C. Humphreys, Arthur Guez, Olivier Tieleman, Laurent Sifre, Théophane Weber, and Timothy Lillicrap. Large-scale retrieval for reinforcement learning. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2206.05314.

[KataGo project, 2026] KataGo project. Other methods implemented in KataGo. <https://github.com/lightvector/KataGo/blob/master/docs/KataGoMethods.md>, 2026. Accessed 2026-10-01.

[Kaufmann and Koolen, 2017] Emilie Kaufmann and Wouter M. Koolen. Monte-Carlo tree search by best arm identification. In *Advances in Neural Information Processing Systems*, 2017. arXiv:1706.02986.

[Lan et al., 2021] Li-Cheng Lan, Meng-Yu Tsai, Ti-Rong Wu, I-Chen Wu, and Cho-Jui Hsieh. Learning to stop: Dynamic simulation Monte-Carlo tree search. In *AAAI Conference on Artificial Intelligence*, 2021. arXiv:2012.07910.

[Leela Chess Zero contributors, 2026] Leela Chess Zero contributors. Lc0 options. <https://github.com/>

416 LeelaChessZero/lc0/wiki/Lc0-options, 2026. Accessed
417 2026-10-01.

418 [Muppidi et al., 2026] Aneesh Muppidi, Firas Darwish,
419 Dylan Cope, João F. Henriques, and Jakob Nico-
420 laus Foerster. Finding the time to think: Learning
421 planning budgets in real-time RL. arXiv preprint
422 arXiv:2606.26463, 2026.

423 [Silver et al., 2016] David Silver, Aja Huang, Chris J.
424 Maddison, Arthur Guez, Laurent Sifre, George
425 van den Driessche, Julian Schrittwieser, Ioannis
426 Antonoglou, Veda Panneershelvam, Marc Lanctot,
427 Sander Dieleman, Dominik Grewe, John Nham,
428 Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap,
429 Madeleine Leach, Koray Kavukcuoglu, Thore Grae-
430 pel, and Demis Hassabis. Mastering the game of Go
431 with deep neural networks and tree search. *Nature*,
432 529:484–489, 2016.

433 [Silver et al., 2017] David Silver, Julian Schrittwieser,
434 Karen Simonyan, Ioannis Antonoglou, Aja Huang,
435 Arthur Guez, Thomas Hubert, Lucas Baker, Matthew
436 Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap,
437 Fan Hui, Laurent Sifre, George van den Driessche,
438 Thore Graepel, and Demis Hassabis. Mastering the
439 game of Go without human knowledge. *Nature*,
440 550:354–359, 2017.

441 [Wang et al., 2023] Tony T. Wang, Adam Gleave, Tom
442 Tseng, Kellin Pelrine, Nora Belrose, Joseph Miller,
443 Michael D. Dennis, Yawen Duan, Viktor Pograbiak,
444 Sergey Levine, and Stuart Russell. Adversarial policies
445 beat superhuman Go AIs. In *International Confer-
446 ence on Machine Learning*, pages 35655–35739, 2023.
447 arXiv:2211.00241.

448 [Wu, 2019] David J. Wu. Accelerating self-play learning
449 in Go. arXiv preprint arXiv:1902.10565, 2019.

450 [Wu, 2026] David J. Wu. KataGo opening books. <https://katagobooks.org/>, 2026. Accessed 2026-10-01.

451

452 [Xiao et al., 2018] Chenjun Xiao, Jincheng Mei, and
453 Martin Müller. Memory-augmented Monte Carlo tree
454 search. In *AAAI Conference on Artificial Intelligence*,
455 2018.

456 [Ye et al., 2022] Weirui Ye, Pieter Abbeel, and Yang
457 Gao. Spending thinking time wisely: Accelerat-
458 ing MCTS with virtual expansions. In *Advances
459 in Neural Information Processing Systems*, 2022.
460 arXiv:2210.12628.